Scaling Reads

什麼是讀取擴展？

讀取擴展（Scaling Reads）處理的是一個很實際的問題：當你的系統從幾百個用戶成長到幾百萬個用戶時，怎麼撐住那些龐大的讀取請求量。寫入是在創造資料，讀取是在消費資料——而讀取流量成長的速度，往往比寫入快得多。這篇講義會介紹在面對海量讀取負載時，常見的架構策略。

問題在哪裡

想像一下 Instagram 的 feed。你打開 App，畫面馬上出現幾十張照片，每一張背後都需要好幾次 database query——要抓圖片的 metadata、用戶資訊、按讚數、留言預覽。光是載入一個 feed，就可能觸發超過 100 次讀取操作。但你一天可能只發一張照片，也就是一次寫入。

這種不對稱非常普遍。一則推文被幾千人讀，一個 Amazon 商品被幾百人瀏覽，YouTube 每天有幾十億次影片播放，但上傳只有幾百萬次。一般來說，讀寫比起碼是 10:1，對內容型應用而言，甚至可以到 100:1 以上。

讀取量一旦增加，你的 database 就會開始撐不住。

更麻煩的是，這通常不是靠改 code 能解決的問題——它是物理限制。CPU 每秒能跑多少指令有上限，記憶體能放多少資料有上限，磁碟 I/O 受限於硬碟轉速或 SSD 的讀寫速度。碰到這些物理瓶頸，加再多程式碼都沒有用。

那要怎麼辦？我們一步一步拆解。

buildnest.org

解法的架構

讀取擴展有一個很自然的演進路徑，從簡單的優化到複雜的分散式系統：

1. 先在 database 內部優化讀取效能
2. 水平擴展你的 database
3. 加入外部 caching layer

讓我們依序介紹每一步。

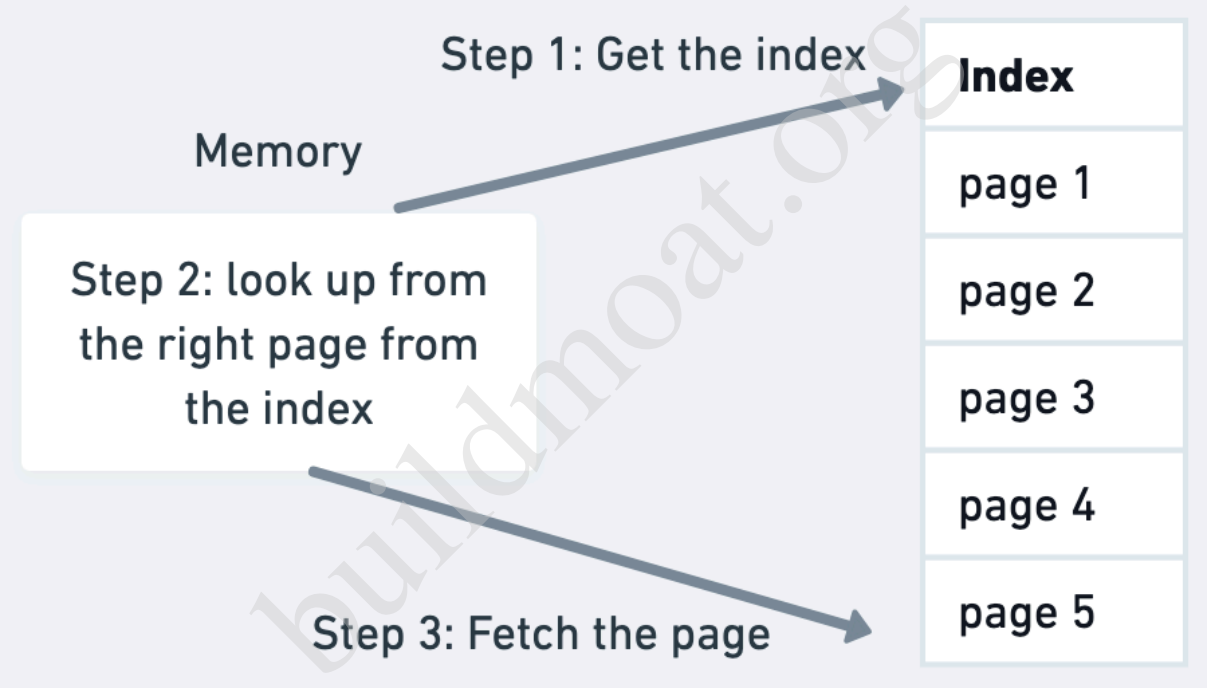
第一步：先把 Database 自己調好

在加更多基礎設施之前，你現有的 database 通常還有很多空間可以擠。大部分的讀取擴展問題，靠適當的調校和聰明的資料建模就能解決。

Index（索引）

Index 本質上是一張排序好的查找表，指向你真實資料裡的那些 row。概念上很像書的索引——你不需要翻遍每一頁去找「database」這個詞，直接翻到書後的索引，它告訴你哪幾頁有提到。

沒有 index 的情況下，database 會做所謂的 full table scan，也就是把每一筆資料都讀過一遍才能找到符合的結果。有了 index，它可以直接跳到對應的 row。這讓複雜度從線性的 O(n) 降到對數的 O(log n)——差別就是從掃描 100 萬筆資料，變成只查大約 20 個 index entry。



Index 有很多種類型，最常見的是 B-tree，適合一般查詢。Hash index 在做精確比對時效果很好；另外還有專門處理全文搜尋或地理查詢的特殊 index。

第一個應該做的事，就是在你常 query、join 或排序的欄位上加 index。 比如你在設計社群媒體 App，用戶常用 hashtag 搜尋貼文，就把 hashtag 欄位加上 index。如果常用 price 排序商品，就 index price 欄位。

你可能看過一些舊資料在警告「太多 index 會讓寫入變慢」。Index 的確有一些 overhead，但在大多數應用裡，這個擔心被過度放大了。現代硬體和 database engine 處理設計合理的 index 效率很高。在面試中，對你的 query pattern 自信地加上 index——index 加太少死掉的應用，遠比 index 加太多的多。

buildmoat.org

硬體升級

有時候答案就是換更好的硬體。無聊，但有效。把機械硬碟換成 SSD，隨機 I/O 可以快個 10 到 100 倍。加更多 RAM，讓更多資料留在記憶體而不是放在磁碟。更快的 CPU 和更多 core，讓你能處理更多併發的 query。

這種方式（俗稱 Vertical Scaling，垂直擴展）不能解決所有問題，但通常是最快幫你買到喘息空間的方法。

> **面試提示**：在面試中提到你會考慮硬體升級是沒問題的，但面試官通常不是在找這個答案，因為它有點在迴避問題的核心。

Denormalization（反正規化）

另一個技巧是把 database 裡的資料組織方式調整得更適合讀取。

Normalization（正規化）是把資料拆分到多張表，避免重複儲存同樣的資訊，節省儲存空間。但代價是 query 變複雜，因為你需要大量的 join 才能把相關資料組合起來。

以電商 database 為例，正規化設計下，users、orders、products 分開放。要顯示一個訂單摘要，你可能要 join 四張表：

```
SELECT u.name, o.order_date, p.product_name, p.price
FROM users u
JOIN orders o ON u.id = o.user_id
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
WHERE o.id = 12345;
```

每秒要回應幾千個訂單頁面時，這樣的 join 就開始變得昂貴。Database 需要跨表查找、比對 foreign key、合併結果。

Denormalization（反正規化）是做相反的事——用空間換速度，允許重複儲存資料。比如建一張 `order_summary` 表，把用戶名稱、商品名稱、價格都直接跟訂單資料放在一起。這樣查一筆訂單只需要一個簡單的單表查詢：

```
SELECT user_name, order_date, product_name, price
FROM order_summary
WHERE order_id = 12345;
```

沒錯，如果一個用戶有多筆訂單，他的名字就會重複存好幾次。但對讀取遠多於寫入的系統而言，這個儲存成本可能完全值得。

buildmoat.org

Normalized

User Table

Terry	Seattle	3-10-24
Bohr	San Francisco	4-04-25
Allen	Taipei	5-10-26

Order Table

Terry	Lamp	3
Terry	Desk	2
Allen	Table	1

Product Table

Table	$100
Desk	$50
Lamp	$10

Denormalized

Order Table

Desk	2	{name: Terry, location: Seattle, date: 3-10-24}	{product: Desk, cost:$100 }
...	...	...	
...	...	...	

你需要在用戶改名時更新多個地方，但這種事很少發生，相比每天幾百萬次的訂單查詢，這個代價是合理的。

Denormalization 是讀寫取捨的典型案例：重複儲存資料讓讀取更快，但讓寫入更複雜。在決定反正規化之前，一定要先想清楚你的讀寫比例。如果寫入也很頻繁，這個複雜度可能不划算。

Materialized view 把這個概念推得更遠——把昂貴的聚合計算預先算好存起來。比起每次載入頁面都重新計算商品平均評分，不如用一個背景排程算好，直接存結果：

```
-- 避免每次頁面載入都跑這個昂貴的 query：
SELECT p.id, AVG(r.rating) as avg_rating
FROM products p
JOIN reviews r ON p.id = r.product_id
GROUP BY p.id;

-- 改用 materialized view 預先計算存好：
CREATE MATERIALIZED VIEW product_ratings AS
SELECT p.id, AVG(r.rating) as avg_rating
FROM products p
JOIN reviews r ON p.id = r.product_id
GROUP BY p.id;
```

第二步：水平擴展你的 Database

當單一 database server 撐到極限，就要加更多 server。這裡開始有趣，也開始複雜。

buildmoat.org

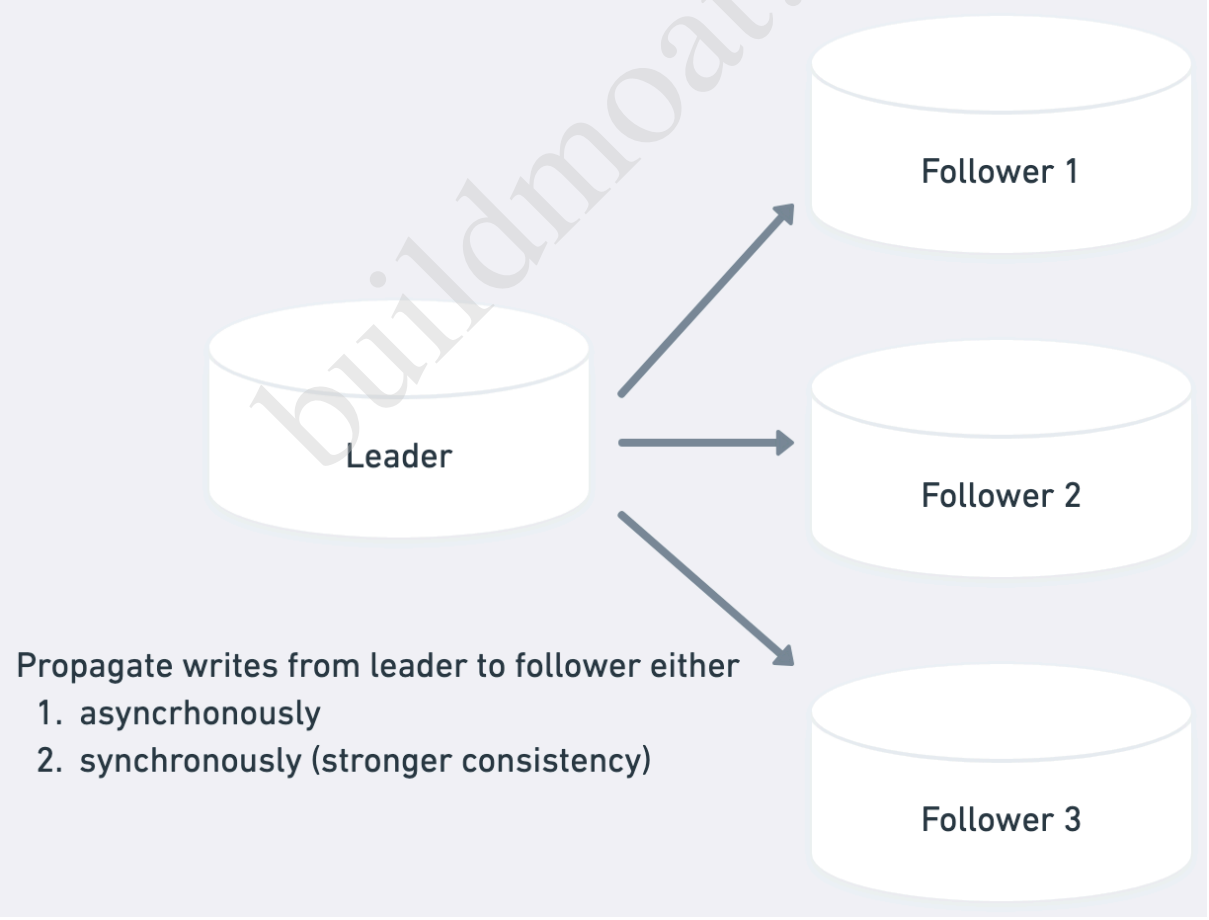
一個粗略的參考點是：當你的讀取請求超過每秒 50,000 到 100,000 次（假設已經有適當的 index），就需要考慮水平擴展或加 cache。這個數字會因你的應用讀取模式、資料模型、硬體等因素而差很多，但在面試裡，這個量級的估計通常已經足夠支撐你的決策。

Read Replica（讀取副本）

超出單一 database 容量的第一個方法，是加 read replica。Read replica 會把 primary database 的資料複製到額外的 server 上。所有寫入都走 primary，但讀取可以打到任何一台 replica，把讀取負載分散開來。

除了解決吞吐量問題，read replica 還帶來一個很棒的副作用：冗余。如果你的 primary database 掛掉，可以把某台 replica 提升為新的 primary，把停機時間降到最低。

標準做法是 Leader-Follower replication：一台 primary（leader）處理寫入，多台 secondary（follower）處理讀取。Replication 可以是同步（synchronous）或非同步（asynchronous）——同步慢但資料一致，非同步快但可能短暫不一致。



buildmoat.org

Replication lag 是核心挑戰。 當你寫入 primary，資料需要時間傳播到 replica。如果用戶寫了什麼東西之後馬上去讀，剛好讀到還沒同步的 replica，就會看不到自己剛剛的改動。

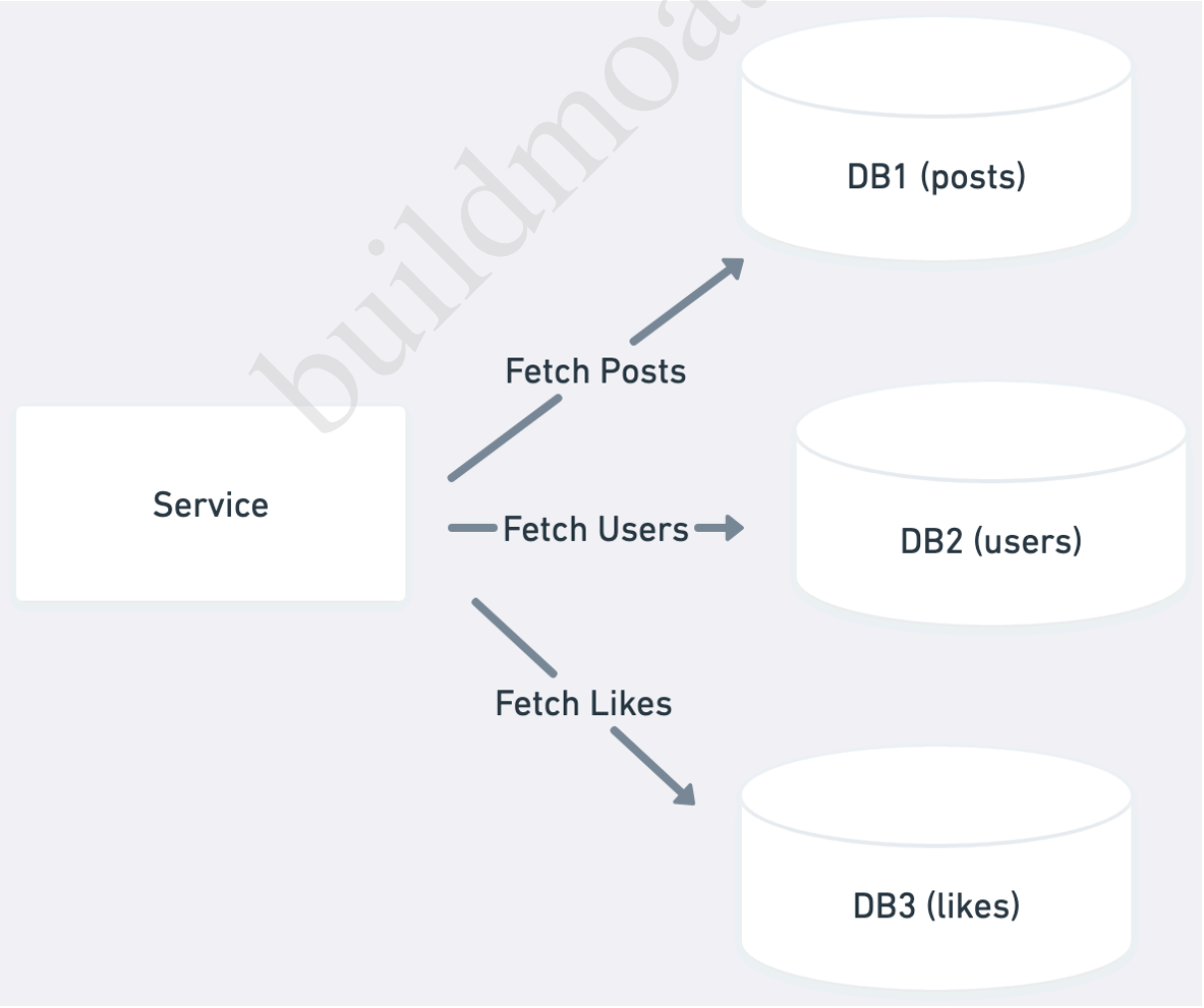
> **面試常考點**：Replication lag 以及同步與非同步 replication 的取捨。同步 replication 確保資料一致，但增加寫入延遲；非同步 replication 更快，但可能短暫出現資料不一致。

Database Sharding（分片）

Read replica 可以分散負載，但每台 database 還是要處理完整的資料集。如果資料量大到連索引良好的查詢也變慢，Sharding 可以幫助把資料分散到多個 database 上。

對讀取擴展而言，Sharding 有兩個主要好處：資料集變小，個別查詢更快；可以把讀取負載分散到多個 database。

Functional sharding（功能分片） 是依照業務功能來拆分資料與database。把用戶資料放一個 user DB，商品資料放在 product DB。這樣一來，查詢用戶 profile 只打 user DB，搜尋商品只打 product DB，各個業務功能的讀寫附載彼此隔離。



buildmoat.org

Geographic sharding（地理分片） 對全球讀取擴展特別有效。美國用戶的資料放在美國的 database，歐洲資料放在歐洲。用戶從附近的 server 讀取，速度更快，也減少了單一 database 的壓力。

不過，Sharding 增加了相當高的運維複雜度，而且它主要是寫入擴展的技術。大多數讀取擴展問題，加 caching layer 更有效也更容易實作。

第三步：加入外部 Caching Layer

當你已經把 database 調好，但讀取效能還是不夠，有兩條路：加 read replica，或是加 cache。兩者都能分散讀取負載，但 cache 通常在讀取密集的情境下效能更好。

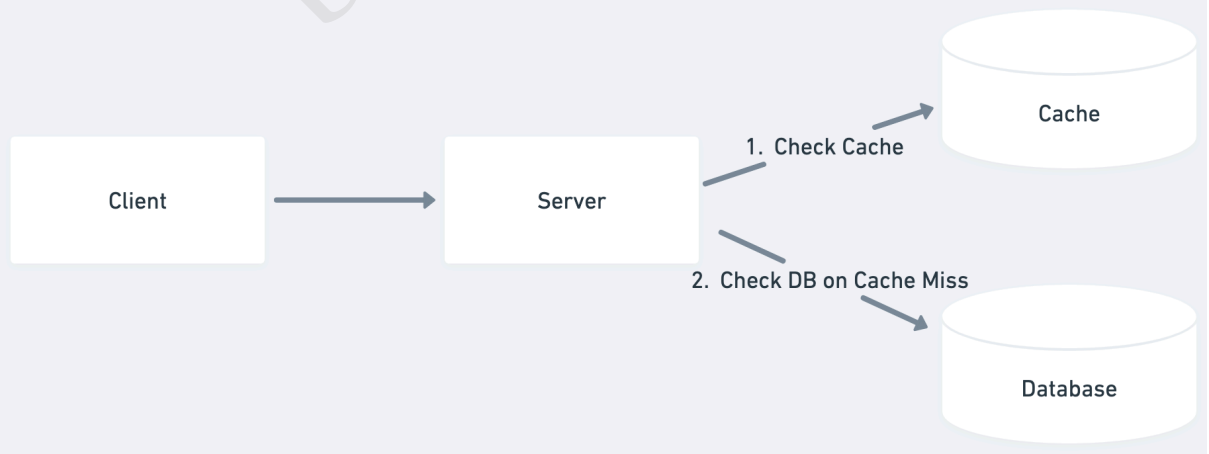
關鍵的觀察是：大多數應用的存取模式都高度偏斜。Twitter 上幾百萬人讀同一則爆紅推文，電商網站幾千人看同一個熱門商品。你的 database 一直在回答完全一樣的問題——而這些答案幾乎不會在兩次請求之間改變。

Cache 利用這個規律，把常被存取的資料放在記憶體裡。Database 需要讀磁碟、執行 query；cache 直接從 RAM 拿預先算好的結果。這個差異轉換成效能就是：cache 回應毫秒以下，database 就算調得很好也要幾十毫秒。

Application-Level Caching（應用層快取）

Redis 或 Memcached 這類 in-memory cache 坐在你的 application 和 database 中間。當 application 需要資料，先查 cache。Hit 了，毫秒以下回來。Miss 了，去查 database，把結果填進 cache 供後續請求使用。

熱門資料自然會留在 cache 裡，冷門資料則等到有人要求時才載入，然後過了 TTL 就失效。像是名人的 profile 因為持續被存取，幾乎永遠在 cache 裡；不活躍用戶的 profile 只有被查到時才會暫時進 cache。



Cache invalidation（快取失效） 是最主要的挑戰——資料改變時，你要確保 cache 不會繼續回傳過時的資料。常見的策略有：

buildnoat.org

- **TTL（Time-to-Live，時間過期）**：給每個 cache entry 設定一個固定的存活時間。實作簡單，但在過期前可能會回傳過時資料。適合更新模式可預期的資料。
- **Write-through invalidation（同步失效）**：寫入 database 時，立刻更新或刪除對應的 cache entry。確保一致性，但對寫入增加了延遲，需要謹慎處理錯誤情況。
- **Write-behind invalidation（非同步失效）**：把失效事件排進 queue 非同步處理。降低寫入延遲，但有一段時間可能會回傳過時資料。
- **Tagged invalidation（標籤失效）**：給 cache entry 加上標籤（例如 `user:123:posts`）。當相關資料改變時，讓所有帶有這個標籤的 entry 失效。對複雜依賴關係很有用，但需要維護標籤關係。
- **Versioned keys（版本化 key）**：在 cache key 裡加入版本號。更新時遞增版本號，舊的 cache entry 自然就失效了。簡單可靠，但需要追蹤版本號。

實際的生產系統通常會組合使用多種策略——用短 TTL（5 到 15 分鐘）作為安全網，對用戶 profile 或庫存這類關鍵資料做主動失效；推薦分數這類次要資料則純靠 TTL 過期就好。

理想情況下，你的 cache TTL 應該由非功能性需求（non-functional requirements）來決定。如果需求說「搜尋結果最多可以 30 秒過時」，你的 TTL 就是 30 秒。用戶 profile 可以接受 5 分鐘的過時，TTL 就設 5 分鐘。這讓 cache 策略的決定有所依據，而不是靠猜。

buildmeat.org

CDN 和 Edge Caching

CDN（Content Delivery Network）把 caching 的概念延伸到你的 data center 之外，延伸到全球各地的 edge location。CDN 最早是拿來放靜態資源的，但現代 CDN 也可以快取動態內容，包括 API response 和 database query 的結果。

地理分散帶來明顯的延遲改善。東京的用戶從東京的 edge server 拿到快取資料，而不是繞一大圈到你在維吉尼亞州的 data center。回應時間從 200 毫秒降到 10 毫秒以下，同時完全消除了 origin server 的負載。

對讀取密集的應用，CDN caching 可以把 origin 的負載降低 90% 以上。商品頁面、用戶 profile、搜尋結果——任何多個用戶會請求的內容——都非常適合放到 edge caching 上。代價是要管理好幾十個 edge location 的 cache invalidation。但效能收益通常值得這份額外工作。

一個重要原則：CDN 只對多個用戶都會存取的資料有意義。不要快取用戶專屬的資料，比如個人偏好設定、私人訊息或帳戶資訊——這些資料只有一個用戶會要，cache

hit rate 根本是零。把 CDN caching 集中在有自然共享模式的內容上：公開貼文、商品目錄、搜尋結果。

什麼時候在面試裡用這些

幾乎每場系統設計面試最後都會談到擴展，而讀取擴展通常是讀取密集應用的起點。我的建議是：看你的每一個 API 請求，找出請求量大的，想辦法優化。從 query 優化開始，然後再考慮 caching 和 read replica。

厲害的面試者會主動發現讀取瓶頸。在畫出 API 設計時，停在那些會被大量打到的 endpoint，說：「這個用戶 profile endpoint 每次有人看 profile 都會被打到。有幾百萬用戶，一天可能有幾十億次讀取。讓我在 deep dive 裡處理這個問題。」

常見面試情境

QR Code Generator：這裡的讀寫比例極度不平衡——一個短網址可能只被縮短一次，但被點擊幾百萬次。這是最適合 caching 的場景。在 Redis 裡積極快取短網址到長網址的對應，不需要設 TTL（短網址不會改變）。搭配 CDN 處理全球流量。實際的 database 只有 cache miss 時才會被查到。

Ticketmaster：演唱會門票開賣時，活動頁面會被瘋狂打。成千上萬個用戶刷新同一個頁面等待座位開放。積極快取活動詳情、場館資訊、座位圖。但要小心——實際的座位庫存不能快取，不然你會超賣。用 read replica 處理瀏覽，write master 才處理實際購票。

News Feed 系統：不管是 Facebook、LinkedIn 還是 Twitter，feed 生成是讀取密集的。為活躍用戶預先計算 feed，快取被追蹤用戶的最新貼文，用聰明的 pagination 避免一次載入整個 feed。關鍵洞察：用戶通常只看前幾則，積極快取最新內容效益最高。

YouTube / 影音平台：影片 metadata 帶來意外高的讀取負載。每次開啟首頁都要 query 推薦影片、頻道資訊、觀看數。積極快取影片 metadata——標題、描述、縮圖都不常變動。觀看數可以接受最終一致性，每幾分鐘更新一次，而不是每次觀看都即時更新。至於影片縮圖這類大型靜態資源，則會走獨立的傳遞路徑，大量透過 CDN 提供給全球使用者。

什麼時候不適合用

寫入密集的系統：如果你在設計 Robotaxi 的位置追蹤，自駕車每幾秒更新一次位置，先聚焦在寫入擴展。讀寫比可能只有 2:1 甚至 1:1。

小規模應用：如果面試官說「設計給 1000 個用戶」，不要直接跳到複雜的 caching 策略。一個索引設計良好的單一 database，輕鬆處理幾千 QPS。展示你的判斷力，解決

buildmoat.org

真實問題，而不是想像中的問題。

強一致性系統：金融交易、庫存管理，或任何有過時資料會造成真實問題的系統，需要謹慎考慮。你也許還是能用這些技術，但要搭配積極的失效和更短的 TTL。

即時協作系統：Google Docs 這類應用需要的是即時更新，而不是讀取擴展。Caching 在每一個按鍵都需要立刻讓其他用戶看到的場景裡，反而是種傷害。

最後記住：讀取擴展是關於減少 database 負載，不只是讓速度更快。如果你的 database 負載沒有問題，只是需要更低的延遲，那是不同的問題，需要不同的解法（例如 edge computing 或 service mesh 優化）。

常見的 Deep Dive 問題

面試官很喜歡測試你對讀取擴展邊緣案例和實際運維挑戰的理解。以下是最常見的追問。

「資料量成長後，query 開始變很慢，怎麼辦？」

你的應用上線時有 10,000 個用戶，query 跑得很快。現在有 1,000 萬個用戶，簡單的查找要 30 秒。Database CPU 一直在 100%，但你沒在做什麼複雜的事，只是找用戶的 email 或抓訂單歷史記錄。

問題在於沒有適當的 index，database 對每個 query 都做 full table scan。有人用 email 登入，database 讀遍全部 1,000 萬筆用戶記錄才找到一筆。每筆 200 bytes，光是掃描就是 2GB 資料——幾百個並發登入同時這樣做，database 就只能一直在讀磁碟了。

多表 join 讓情況更糟。沒有 index 的情況下，抓用戶訂單要掃整張 users 表，再掃整張 orders 表比對。幾十億次 row 比對，就只為了一個簡單的查找。

解法很直接——**在你常查詢的欄位上加 index**。email 欄位加上 index，1,000 萬次掃描變成幾次 index 查找：

```
-- 之前：Full table scan
EXPLAIN SELECT * FROM users WHERE email = 'user@example.co
m';
-- Seq Scan on users (cost=0.00..412,000.00 rows=1)

-- 加上 index
CREATE INDEX idx_users_email ON users(email);

-- 之後：Index scan
```

buildmoat.org

```
EXPLAIN SELECT * FROM users WHERE email = 'user@example.co
m';
-- Index Scan using idx_users_email (cost=0.43..8.45 rows=
1)
```

複合查詢要注意 index 的欄位順序。如果用戶用 status AND created_at 搜尋，`(status, created_at)` 的 index 可以幫到「只用 status 過濾」和「同時用兩個欄位過濾」的查詢，但對「只用 created_at 過濾」沒有幫助。

在面試中畫出 database schema 時，順帶說明你會在哪些欄位加 index，展示你有在主動思考這個問題。

「如果幾百萬個請求同時要存同一份快取資料，怎麼辦？」

一個名人在你的平台發了貼文，幾百萬用戶同時要讀。你的 cache server 平時每秒處理 50,000 個請求，現在對同一個 key 收到 500,000 個請求每秒。Cache 開始 timeout，請求失敗，網站掛掉——就因為讀取流量。

問題是傳統 caching 假設負載會分散在很多 key 上。當每個人都要同一個 key，這個假設就破了。你的 cache server 有有限的 CPU 和網路容量，就算資料在記憶體裡，每秒序列化 500,000 次然後傳出去，也能把任何一台 server 搞垮。

第一個解法是 request coalescing（請求合併）——把對同一個 key 的多個請求合併成一個請求。當 backend 撐不住大家同時問同一件事時，這個模式很有用：

```
# Request coalescing 模式
class CoalescingCache:
    def __init__(self):
        self.inflight = {}  # key -> Future

    async def get(self, key):
        # 如果已經有請求正在抓這個 key，就等它
        if key in self.inflight:
            return await self.inflight[key]

        # 沒有進行中的請求，我們來抓
        future = asyncio.Future()
        self.inflight[key] = future

        try:
            value = await fetch_from_backend(key)
```

guldnot.org

```
            future.set_result(value)
            return value
        finally:
            del self.inflight[key]
```

Request coalescing 的重點是：它把 backend 的請求量從「無限大」（每個用戶各自打）降到剛好 N 個，N 是你 application server 的數量。就算幾百萬用戶同時想要同一份資料，你的 backend 只收到 N 個請求——每台 server 各一個。

當 coalescing 對極端負載還不夠用，就需要分散負載本身了。**Cache key fanout（key 扇出）** 把一個熱 key 分散到多個 cache entry。與其把名人的貼文存在一個 key 下，複製十份分別存到十個不同的 key。Client 隨機挑一個，把 500,000 requests/sec 分散到十個 key，每個各 50,000——這才是 cache server 能處理的量。做法就是在 key 後面加隨機數，比如 `feed:taylor-swift:1`、`feed:taylor-swift:2`，諸如此類。

Fanout 的代價是記憶體使用增加和 cache 一致性變複雜——同樣的資料存了好幾份，失效時要清掉所有副本。但對讀取密集的場景，這個冗余換來服務不掛掉是值得的。

「多個請求同時要重建過期的 cache entry，怎麼辦？」

你的首頁資料每秒有 100,000 個請求，全部都從 cache 拿。Cache entry 有 1 小時的 TTL。當那一小時到了，這 100,000 個請求在同一瞬間全部看到 cache miss，每一個都試著去查 database。

你的 database 平時能處理大概每秒 1,000 個查詢（正常 cache miss 量），現在突然被打了 100,000 個一模一樣的查詢。像是你自己的應用對自己做了一次 DDoS 攻擊。Database 開始癱瘓甚至當機，整個網站跟著崩潰。

這就是 cache stampede（快取雪崩）——因為 cache 過期是二元的，這一秒資料還在，下一秒就什麼都沒有。熱門的 entry 製造更大的雪崩。如果重建一個 entry 需要 join 10 張表或呼叫外部 API，你剛剛並行發動了 100,000 個昂貴的操作。

第一個方法是用 distributed lock（分散式鎖）序列化重建。第一個發現 cache miss 的請求拿到 lock 去重建，其他請求等待重建完成。這保護了 backend，但有嚴重的缺點：如果重建失敗或花太長時間，幾千個請求就等到 timeout。你需要複雜的 timeout 處理和 fallback 邏輯，在高負載下很脆弱。

更聰明的方法是 probabilistic early refresh（機率性提前刷新）——在背景裡刷新 cache，同時繼續提供快取資料。Entry 剛建好時，請求直接使用它。但隨著 entry 越來越舊，每個請求觸發背景刷新的機率越來越高。假設 entry 在 60 分鐘後過期：第 50 分鐘時，某個請求可能有 1% 的機率觸發刷新；第 55 分鐘時，機率提高到 5%；第 59 分鐘時，可能是 20%。這樣一來，不是在第 60 分鐘有 100,000 個請求同時打

buildroot.org

database，而是把這些刷新分散在最後 10 到 15 分鐘裡。大多數用戶還是拿到快取資料，只有少數「不幸」的請求觸發刷新。

對最重要的快取資料，上面兩種方法可能都不夠好。你承受不起任何雪崩，也不想等機率性刷新。這時可以用**背景主動刷新**——持續在過期前就更新 entry。首頁 cache 每 50 分鐘就主動更新，保證用戶永遠不需要觸發重建。代價是基礎設施複雜度，以及可能刷新了根本沒人要的資料——但對你最熱門的內容，這個保險是值得的。

「資料更新需要立刻反映出來時，cache invalidation 怎麼處理？」

對某些資料來說，最終一致性是不可接受的。如果活動主辦方更新了場館地址，出席者不應該因為 cache 還沒過期就繼續看到舊地址。但實際上，你的資料可能被快取在多個層次：Redis、CDN edge、甚至 browser cache。要失效所有這些 cache，出了名的困難。

常見的直覺做法是**寫入後刪除 cache entry**。聽起來簡單，但問題多：

- 你要刪的是哪些 cache？Application cache、CDN、browser？
- 如果某個失效請求失敗了怎麼辦？
- 如果刪除舊值之後、新值寫入之前，剛好有請求進來，它就會把過時的資料重新算一遍存進 cache，這是個 race condition。

更好的方法是 cache versioning（快取版本化），也叫做 cache key versioning。每次資料變動時，不刪舊的 cache entry，而是換一個 cache key，讓舊 entry 自然失效。

運作方式如下：

每筆記錄在 database 裡有一個版本號。每次更新這筆記錄，版本號在同一個 transaction 裡遞增。

讀取時：

1. 讀取版本號（從快取的 version key 讀取，或 fallback 到 database）
2. 用版本號組合 cache key，例如 `event:123:v42`
3. 用這個帶版本號的 key 讀取資料
4. Cache miss 的話，從 database 抓資料，用同樣的帶版本號 key 寫回 cache

寫入時：

1. 開啟 DB transaction
2. 更新資料
3. 在同一個 transaction 裡把版本號遞增（`version = version + 1`）

buildnoat.org

4. Commit 之後，用新版本號的 key 寫入新資料（`event:123:v43`）

舊的 cache entry（像 `event:123:v42`）永遠不會被刪除——它只是在版本改變後變得無法存取。CDN 和 browser cache 因為 version 是 URL 或 cache key 的一部分，舊版本自然就過期了。

Cache versioning 的美妙之處在於它從根本上迴避了整類的 invalidation 問題。沒有 race condition，因為「遲到的寫入者」無法覆蓋新資料——database 會強制用新的版本號。不需要擔心局部失效，因為你不是在刪 cache，而是繞過它。在並發下完全安全，因為版本號的改變在 database 裡是原子操作，cache key 永遠不會衝突。

範例 cache key：

```
event:123:v42      // 更新前
event:123:v43      // 更新後
```

讀取方在版本改變後自動轉向 `v43`——不需要廣播失效訊號，不需要猜測要刪哪些 cache。

當然，這個方法也有自己的取捨。每次請求需要兩次 cache 查找——一次查版本號，一次查實際資料，比單次 cache hit 多了一點延遲。舊版本的 cache entry 會隨著時間累積，因為你從不主動刪除它們，所以你應該設合理的 TTL 讓過時 entry 自動清理。這個模式對單一 entity 的 cache（例如用戶 profile、商品詳情）最有用，對搜尋結果或 feed 這類計算出來的資料幫助不大，因為它們的失效本來就比較複雜。

當 versioning 不實用時——比如你在快取搜尋結果或聚合資料——你需要明確的失效策略。另一個方法是**deleted items cache（已刪除項目快取）**：維護一個小型的「最近被刪除、隱藏或修改的項目」快取。比起讓所有包含某則被刪貼文的 feed cache 都失效，你只需要在這個小 cache 裡記錄最近刪除的貼文 ID。提供 feed 時，先查這個小 cache，把符合的項目過濾掉。這讓你能立刻回傳「大致正確」的快取 feed，同時在背景慢慢失效那些更大、更複雜的 cache 結構。Deleted items cache 可以很小（只記近期的刪除），查起來很快，對內容管理和隱私變更這類場景非常有效率。

對於有 CDN caching 的全球系統，失效變得更複雜——你不是在清一個 cache，而是可能幾十個全球 edge location。CDN API 可以幫忙，但傳播需要時間。關鍵更新可以用 cache header 完全阻止 CDN 快取，用一致性換效能。次要資料可以在 edge 設短 TTL，在 application cache 設長 TTL。

不同資料有不同的一致性需求。用戶 profile 也許 5 分鐘的過時可以接受，但活動場館需要即時更新。根據這些需求設計你的 caching 策略，而不是對所有東西用同一套方法。

quinnat.org

總結

讀取擴展是系統設計面試中最常見的擴展挑戰，幾乎在所有內容型應用——從社群媒體到電商——都會遇到。要記住的核心概念是：讀取流量的成長速度遠快於寫入，而物理限制終究會贏——當你在服務幾百萬個並發用戶時，沒有任何聰明的程式碼能突破硬體的極限。

成功的路徑有清楚的演進順序：先用適當的 index 和 denormalization 在 database 內部優化，再用 read replica 水平擴展，最後加入 caching layer 追求極致效能。很多面試者會直接跳到複雜的分散式 caching，而沒有先把更簡單的方法用盡。先從你擁有的開始——現代 database 在正確的 index 和配置下，能承受遠超過大多數工程師預期的負載。

在面試裡，展示你同時理解每個方法的效能收益和運維複雜度。展示你知道什麼時候要對不常改變的內容積極快取，什麼時候要靠 read replica 讓資料保持新鮮——這才是真正有深度的系統設計思維。

buildmoat.org